



Instituto Tecnológico de Mazatlán

Estación barométrica con sensor BMP280 por I2C

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Estación barométrica con sensor BMP280 por I2C

1. Descripción del proyecto

Esta práctica (P3.3) consiste en construir una pequeña estación barométrica con un sensor BMP280 conectado a un Arduino UNO R4 WiFi mediante el bus I2C. El sensor entrega la temperatura y la presión atmosférica, y a partir de la presión el programa calcula la altitud aproximada respecto al nivel del mar.

Las lecturas se envían cada segundo por el puerto serie a 115200 baudios con el formato etiqueta:valor, de modo que pueden verse como texto en el Monitor Serie o como gráficas en el Serial Plotter. Al mismo tiempo, la matriz de LEDs de 12x8 integrada en la placa alterna cada 2.5 segundos entre la temperatura en grados Celsius y la altitud en metros.

Todo el programa está temporizado con `millis()`, sin utilizar `delay()`, e incluye la detección automática del sensor, la validación de las lecturas y el reintento de conexión cuando el sensor no responde.

2. Objetivos

- Conectar y configurar un sensor BMP280 mediante el protocolo I2C.
- Identificar el dispositivo en el bus por su dirección (0x76 o 0x77) y por su registro de identificación (Chip ID).
- Obtener la temperatura y la presión atmosférica, y calcular la altitud a partir de la presión.
- Mostrar las lecturas en el Monitor Serie y graficarlas con el Serial Plotter.
- Mostrar información en la matriz de LEDs integrada del Arduino UNO R4 WiFi.
- Organizar varias tareas periódicas con `millis()` sin bloquear el programa.
- Detectar lecturas inválidas o la desconexión del sensor y recuperar la comunicación automáticamente.

3. Herramientas y material utilizado

Hardware:

- Arduino UNO R4 WiFi (con matriz de LEDs 12x8 integrada)
- Módulo sensor BMP280 (presión y temperatura, I2C)
- Protoboard
- Cables de conexión
- Cable USB-C

Software:

- Arduino IDE
- Librerías Adafruit BMP280 Library y Adafruit Unified Sensor
- Librerías ArduinoGraphics y Arduino_LED_Matrix
- Librería Wire
- Monitor Serie y Serial Plotter

4. Diagrama

El sensor se conecta al Arduino UNO R4 WiFi con solo cuatro cables. El módulo se alimenta con 3.3 V y comparte la tierra con la placa; la línea SDA del sensor se conecta al pin A4 (SDA) y la línea SCL al pin A5 (SCL), que corresponden al bus Wire. Los pines CSB y SDO del módulo se dejan sin conectar. Si se utiliza el conector Qwiic de la placa, el bus que se emplea es Wire1 y en el código basta con cambiar la constante USE_QWIIC a 1.

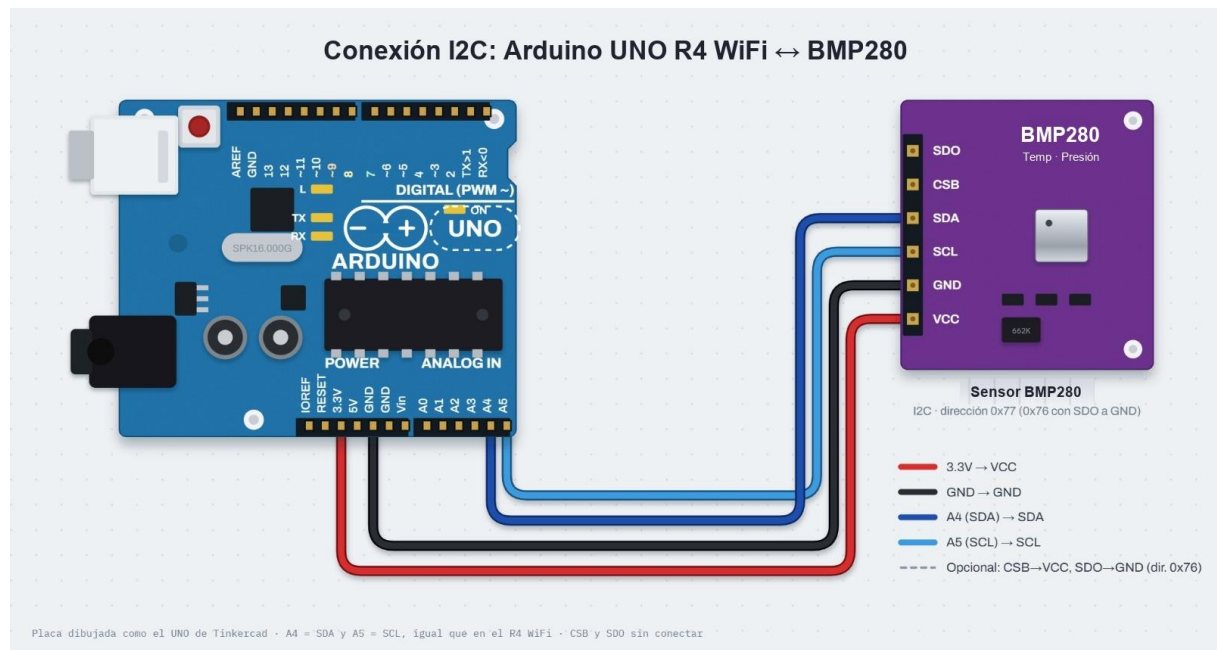


Diagrama de conexión del sensor con el Arduino UNO R4 WiFi.

5. Código

El siguiente código busca el sensor en el bus I2C, lo configura y después ejecuta dos tareas periódicas: la lectura del sensor cada segundo y la actualización de la matriz de LEDs cada 2.5 segundos:

```
/*
 * =====
 * Estación Barométrica BMP280 - Arduino UNO R4 WiFi
 * Sistemas Programables (SCC-1023) - Práctica P3.3
 * =====
 */
```

```

* Lee temperatura, presión y altitud del BMP280 por I2C,
* sin delay() (temporización con millis()).
* - Monitor/Serial Plotter a 115200 baudios
* - Matriz LED 12x8: alterna temperatura (°C) y altitud (m)
*
* Librerías (Gestor de librerías):
*   - Adafruit BMP280 Library
*   - Adafruit Unified Sensor
*   - ArduinoGraphics
*
* CONEXIÓN:
*   BMP280 VCC -> 3.3V      BMP280 GND -> GND
*   BMP280 SDA -> SDA      BMP280 SCL -> SCL
*
* ¿Conector Qwiic? Cambia USE_QWIIC a 1 (usa Wire1).
* =====
*/

#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BMP280.h>
#include "ArduinoGraphics.h" // SIEMPRE antes de Arduino_LED_Matrix.h
#include "Arduino_LED_Matrix.h"

// ----- Configuración -----
#define USE_QWIIC 0 // 0 = pines SDA/SCL, 1 = conector Qwiic

#if USE_QWIIC
  #define BUS_I2C Wire1
#else
  #define BUS_I2C Wire
#endif

const float SEALEVEL_HPA = 1005.0; // presión a nivel del mar del día
const unsigned long T_LECTURA = 1000; // ms entre lecturas
const unsigned long T_MATRIZ = 2500; // ms entre cambios en la matriz
const unsigned long T_REINTENTO = 5000; // ms entre reintentos si no hay sensor

const uint8_t REG_CHIP_ID = 0xD0;
const uint8_t ID_BMP280 = 0x58;
const uint8_t ID_BME280 = 0x60;

// ----- Objetos y estado -----
Adafruit_BMP280 bmp(&BUS_I2C);
ArduinoLEDMatrix matrix;

bool sensorOK = false;
bool datosValidos = false;
uint8_t direccion = 0;

float tempC = 0, presionHpa = 0, altitudM = 0;

```

```

unsigned long tLectura = 0, tMatriz = 0, tReintento = 0;
uint8_t pantalla = 0;    // 0 = temperatura, 1 = altitud

// ----- Funciones auxiliares -----
uint8_t leerChipId(uint8_t addr) {
    BUS_I2C.beginTransaction(addr);
    BUS_I2C.write(REG_CHIP_ID);
    if (BUS_I2C.endTransmission(false) != 0) return 0;
    if (BUS_I2C.requestFrom((int)addr, 1) != 1) return 0;
    return BUS_I2C.read();
}

bool iniciarSensor() {
    const uint8_t candidatos[] = {0x76, 0x77};

    for (uint8_t addr : candidatos) {
        uint8_t id = leerChipId(addr);
        if (id == 0) continue;

        Serial.print("Dispositivo en 0x");
        Serial.print(addr, HEX);
        Serial.print(" | Chip ID: 0x");
        Serial.println(id, HEX);

        if (id == ID_BME280) {
            Serial.println("ATENCIÓN: es un BME280. Usa el sketch con
Adafruit_BME280.");
            return false;
        }
        if (id == ID_BMP280 && bmp.begin(addr, ID_BMP280)) {
            direccion = addr;
            // Modo normal recomendado por Bosch para estación meteorológica de
interior:
            // temp x2, presión x16, filtro IIR x16, espera 500 ms entre mediciones
            bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
                           Adafruit_BMP280::SAMPLING_X2,    // temperatura
                           Adafruit_BMP280::SAMPLING_X16,    // presión
                           Adafruit_BMP280::FILTER_X16,
                           Adafruit_BMP280::STANDBY_MS_500);

            return true;
        }
    }
    return false;
}

// Muestra hasta 3 caracteres en la matriz (fuente 4x6 -> 12 columnas)
void mostrarMatriz(const String &txt) {
    matrix.beginDraw();
    matrix.stroke(0xFFFFFFFF);
    matrix.textFont(Font_4x6);
    matrix.beginText(0, 1, 0xFFFFFF);
    matrix.print(txt);
}

```

```

    matrix.endText();
    matrix.endDraw();
}

void leerSensor() {
    tempC      = bmp.readTemperature();
    presionHpa = bmp.readPressure() / 100.0F;
    altitudM   = bmp.readAltitude(SEALEVEL_HPA);

    // Si se desconecta el sensor, la librería devuelve NaN o valores absurdos
    if (isnan(tempC) || isnan(presionHpa) || presionHpa < 300 || presionHpa > 1100)
    {
        Serial.println("Error: lectura invalida. Revisa el cableado.");
        datosValidos = false;
        sensorOK = false;
        mostrarMatriz("ERR");
        return;
    }
    datosValidos = true;

    // Formato etiqueta:valor -> compatible con Serial Plotter
    Serial.print("Temp_C:");          Serial.print(tempC, 2);
    Serial.print(",Presion_Atm_hPa:"); Serial.print(presionHpa, 2);
    Serial.print(",Altura_nivel_mar_m:"); Serial.println(altitudM, 1);
}

// ----- Programa principal -----
void setup() {
    Serial.begin(115200);
    unsigned long t0 = millis();
    while (!Serial && millis() - t0 < 3000) { } // espera USB máx. 3 s

    matrix.begin();
    BUS_I2C.begin();

    Serial.println("\n=== Estacion BMP280 - UNO R4 WiFi ===");
    Serial.println(USE_QWIIC ? "Bus: Wire1 (Qwiic)" : "Bus: Wire (SDA/SCL)");

    sensorOK = iniciarSensor();
    if (sensorOK) {
        Serial.print("BMP280 listo en 0x");
        Serial.println(direccion, HEX);
    } else {
        Serial.println("No se encontro el BMP280. Reintentando cada 5 s...");
        mostrarMatriz("ERR");
    }
}

void loop() {
    unsigned long ahora = millis();

    // Reintento de conexión si el sensor no está disponible

```

```

if (!sensorOK) {
    if (ahora - tReintento >= T_REINTENTO) {
        tReintento = ahora;
        sensorOK = iniciarSensor();
        if (!sensorOK) mostrarMatriz("ERR");
    }
    return;
}

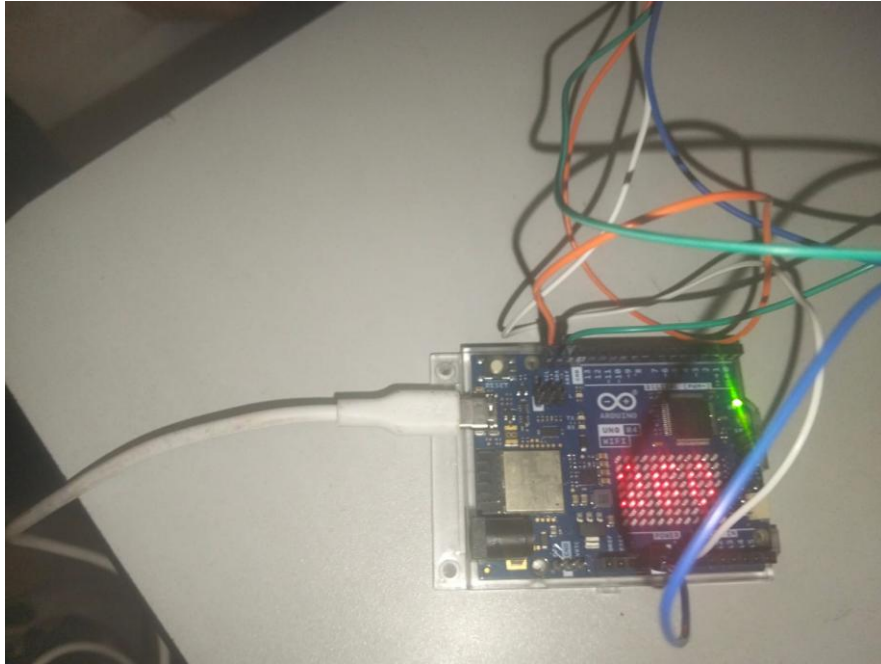
// Tarea 1: lectura del sensor
if (ahora - tLectura >= T_LECTURA) {
    tLectura = ahora;
    leerSensor();
}

// Tarea 2: actualización de la matriz LED
if (datosValidos && ahora - tMatriz >= T_MATRIZ) {
    tMatriz = ahora;
    if (pantalla == 0) {
        mostrarMatriz(String((int)round(tempC)) + "C");
    } else {
        mostrarMatriz(String((int)round(altitudM)) + "m");
    }
    pantalla = (pantalla + 1) % 2;
}
}

```

6. Montaje físico

Como parte de la práctica se realizó el armado físico del circuito, replicando las conexiones del diagrama y verificando de forma presencial la comunicación con el sensor y la información mostrada en la matriz de LEDs.



Montaje físico del circuito con la matriz de LEDs mostrando una lectura.

7. Metodología

Primero se instalaron en el Arduino IDE las librerías necesarias y se conectó el módulo BMP280 al Arduino UNO R4 WiFi, alimentándolo con 3.3 V y uniendo las líneas SDA y SCL al bus I2C de la placa.

En el programa se implementó una función que recorre las dos direcciones posibles del sensor (0x76 y 0x77) y lee el registro 0xD0, que contiene el identificador del chip. Si el valor es 0x58 se trata de un BMP280 y se inicializa la librería; si es 0x60 se trata de un BME280 y se avisa por el Monitor Serie que se requiere otra librería.

Una vez detectado, el sensor se configuró en modo normal con sobremuestreo x2 para la temperatura, x16 para la presión, filtro IIR x16 y un tiempo de espera de 500 ms entre mediciones, que es la configuración recomendada por el fabricante para una estación meteorológica de interior.

Dentro del loop() se programaron dos tareas independientes controladas con millis(): la lectura del sensor cada 1000 ms, que imprime los valores por el puerto serie, y el cambio de pantalla de la matriz de LEDs cada 2500 ms. Además, cuando el sensor no está disponible, el programa vuelve a buscarlo cada 5000 ms.

Finalmente se abrió el Monitor Serie a 115200 baudios para observar las lecturas, y se comprobó en la matriz de LEDs la alternancia entre temperatura y altitud.

8. Explicación del funcionamiento

El BMP280 es un sensor digital de presión barométrica y temperatura. Funciona como esclavo dentro del bus I2C: el Arduino, que es el maestro, lo llama por su dirección y le solicita el contenido de sus registros. La dirección depende del pin SDO del módulo: 0x76 si está conectado a GND y 0x77 si está conectado a VCC, por lo que el programa prueba ambas.

Para confirmar qué dispositivo respondió, la función leerChipId() escribe el número de registro 0xD0 y después pide un byte. Se utiliza endTransmission(false) para no liberar el bus entre la escritura y la lectura (condición de inicio repetido). Si el esclavo no contesta, la función devuelve 0 y esa dirección se descarta.

La librería entrega la temperatura en grados Celsius y la presión en pascales, por lo que esta última se divide entre 100 para expresarla en hectopascales. La altitud no se mide directamente: se calcula con la fórmula barométrica $h = 44330 \cdot (1 - (P / P_0)^{(1/5.255)})$, donde P es la presión medida y P0 la presión a nivel del mar. Por esa razón el resultado depende del valor de la constante SEALEVEL_HPA, que debe ajustarse a la presión del día para obtener una altitud realista.

Cada lectura se valida antes de usarse. El BMP280 mide entre 300 y 1100 hPa, así que cualquier valor fuera de ese intervalo, o un resultado que no sea un número (NaN), se interpreta como una falla de cableado: se muestra ERR en la matriz, se informa por el Monitor Serie y el programa pasa al modo de reintento.

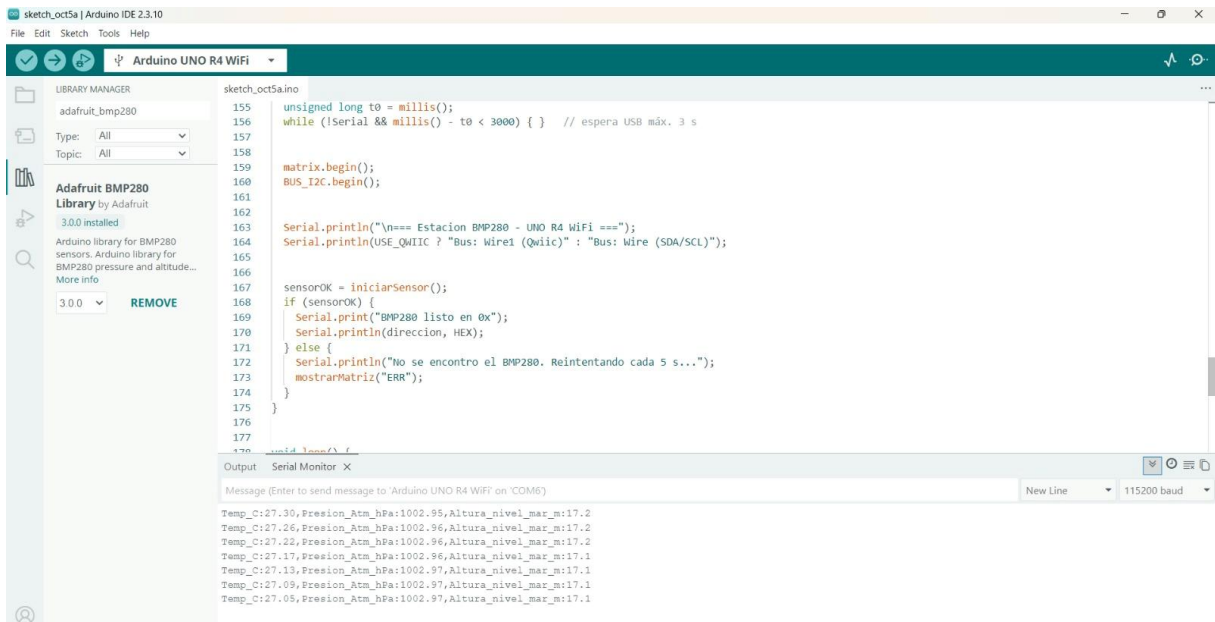
La salida serie utiliza el formato etiqueta:valor separado por comas (Temp_C, Presion_Atm_hPa y Altura_nivel_mar_m). Con ese formato el Serial Plotter reconoce tres series distintas y las grafica con su nombre, sin necesidad de modificar el programa.

La matriz de LEDs se maneja con las librerías ArduinoGraphics y Arduino_LED_Matrix. Se usa la fuente de 4x6 píxeles, con la que caben tres caracteres en las 12 columnas de la matriz; por eso la temperatura y la altitud se redondean a enteros antes de mostrarse, seguidas de la letra C o m.

Como ninguna tarea utiliza delay(), el Arduino nunca queda detenido: en cada vuelta del loop() se compara el tiempo actual con el de la última ejecución de cada tarea y solo se ejecuta la que ya cumplió su intervalo. Así la lectura del sensor y la matriz trabajan con periodos distintos sin interferir entre sí.

9. Monitor Serie

Con el sensor funcionando, el Monitor Serie muestra cada segundo una línea con la temperatura, la presión atmosférica y la altitud calculada. En la misma captura se observa la librería Adafruit BMP280 Library instalada en el Arduino IDE y la velocidad de 115200 baudios:



Monitor Serie mostrando temperatura, presión y altitud.

10. Análisis de resultados

Durante las pruebas el Arduino se comunicó correctamente con el sensor por el bus I2C y envió una lectura por segundo con el formato esperado: Temp_C, Presion_Atm_hPa y Altura_nivel_mar_m.

En la captura del Monitor Serie la temperatura registrada estuvo entre 27.05 °C y 27.30 °C, disminuyendo de forma gradual unas cuatro centésimas de grado en cada lectura. La presión atmosférica se mantuvo prácticamente constante, entre 1002.95 hPa y 1002.97 hPa, lo que muestra la estabilidad que aportan el sobremuestreo y el filtro IIR configurados en el sensor.

La altitud calculada fue de 17.1 a 17.2 metros. Este valor no se mide directamente, sino que resulta de comparar la presión medida (1002.96 hPa) con la presión de referencia a nivel del mar definida en el código (1005.0 hPa): una diferencia de aproximadamente 2 hPa equivale a unos 17 metros. Por eso la altitud depende de que la constante SEALEVEL_HPA corresponda a la presión real del día.

En el montaje físico la matriz de LEDs mostró las lecturas mientras el Monitor Serie seguía recibiendo datos cada segundo, lo que muestra que las dos tareas se ejecutan de forma independiente con millis(), sin que una detenga a la otra.

11. Conclusiones

Gabriel

En esta práctica comprendí que un sensor I2C se maneja de la misma forma que los esclavos que programamos en la práctica anterior: tiene una dirección y registros que el maestro consulta. Leer el Chip ID antes de iniciar la librería me pareció una buena forma de saber exactamente qué dispositivo está conectado y de evitar errores difíciles de encontrar.

Javier

Aprendí que la altitud que entrega el programa no es una medición directa, sino un cálculo a partir de la presión, y que por eso cambia con el clima y con la presión de referencia que se utilice. También reforcé el uso de `millis()` para ejecutar tareas con periodos distintos sin que una detenga a la otra.

Rosa

Esta práctica me permitió ver cómo un mismo dato puede presentarse de varias maneras: como texto en el Monitor Serie, en un formato que también puede graficarse, y en la matriz de LEDs de la placa. Además, comprendí la importancia de validar las lecturas y de reintentar la conexión para que el sistema no se quede detenido cuando el sensor falla.